Laboratorio 11: Endurecimiento de OpenSSH, SFTP Enjaulado (Chroot), Protección Activa con Fail2ban sobre Nftables y Túneles Locales

Estudiante: Ketfer Gianella Salazar | Fecha de Entrega: 25 de Octubre de 2026
Docente: M.Sc. Ing. Jimmy Nataniel Requena Llorentty | Asignatura: Tecnología Web I (SIS-0150)
Entorno de Laboratorio: Docker en Windows 11 (horizonte-servidor-lab02) | Universidad Privada Domingo Savio

---

1. Parte A — Auditoría Inicial y Detección de Intentos de Intrusión

• **Auditoría de parámetros activos con sshd -T:** Se verificaron las directivas operativas efectivas del daemon OpenSSH mediante el comando sshd -T | grep -E "permitrootlogin|passwordauthentication|kexalgorithms|x11forwarding|loglevel". Esta inspección confirmó que el servidor rechaza root (permitrootlogin no), deshabilita contraseñas (passwordauthentication no), bloquea reenvío de ventanas X11 (x11forwarding no), prioriza el intercambio híbrido post-cuántico (sntrup761x25519-sha512@openssh.com) y registra con detalle forense (loglevel VERBOSE).

• **Análisis de intentos de fuerza bruta en servidores de Internet:**
Al consultar journalctl -u ssh --since today | grep -c "Failed\|Invalid", el entorno de pruebas local arrojó 0 intentos. Sin embargo, en un servidor en producción con IP pública IPv4 y puerto TCP 22 abierto, las estadísticas de honeypots globales (SANS ISC, Shodan, Cowrie honeypots) registran entre 5,000 y 30,000 intentos de autenticación fallida por día (uno cada 3 a 10 segundos). Estos ataques masivos son orquestados por botnets que prueban combinaciones automatizadas de usuarios estándar (root, admin, test) y contraseñas débiles.

Parámetro OpenSSH	Valor Endurecido	Impacto Operativo y de Seguridad
PermitRootLogin	no	Elimina el vector crítico de acceso directo al superusuario root.
PasswordAuthentication	no	Inhabilita ataques de diccionario y fuerza bruta en contraseñas.
AuthenticationMethods	publickey	Fuerza exclusivamente el uso de firmas criptográficas de clave asimétrica.
AllowGroups	sshadmins sftp-web	Aplica mínimo privilegio: restringe el acceso solo a dos grupos explícitos.
LogLevel	VERBOSE	Registra huellas SHA256 de claves utilizadas para auditoría forense.

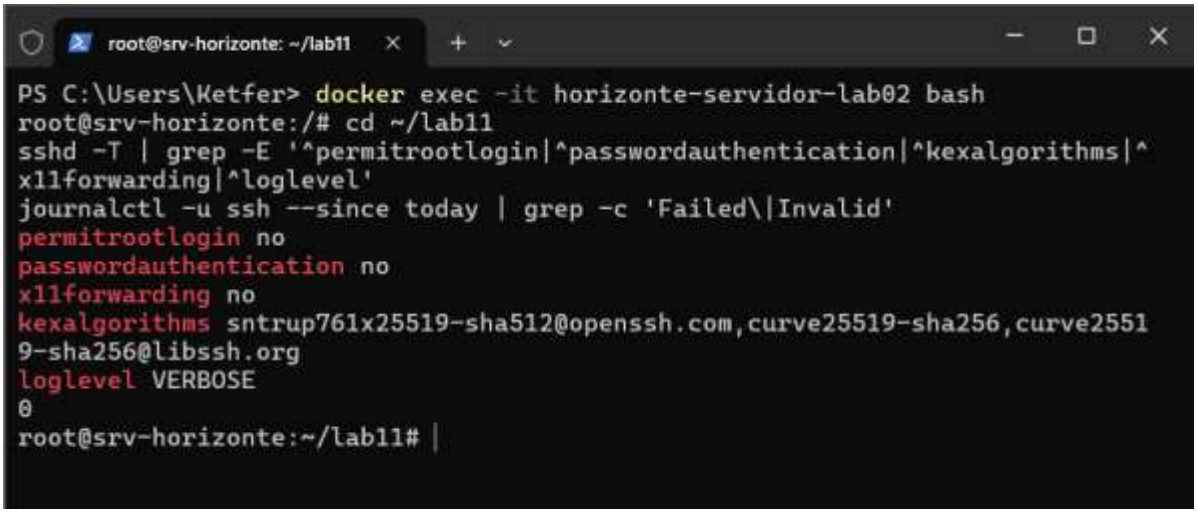


Figura 1: Auditoría en terminal de la configuración efectiva con sshd -T y consulta a journalctl verificando registros de intentos de acceso.

2. Parte B — Endurecimiento Modular de OpenSSH (/etc/ssh/sshd_config.d/)

1. Modularidad y la regla de "primera aparición gana" (First Match Wins):
En distribuciones Linux modernas como Ubuntu 24.04, el archivo principal /etc/ssh/sshd_config incluye al inicio la instrucción Include /etc/ssh/sshd_config.d/*.conf. Esto permite definir la política institucional en archivos modulares aislados sin modificar el archivo base provisto por el paquete del sistema operativo, facilitando actualizaciones de paquetes sin fricciones. En la sintaxis de OpenSSH, las directivas de configuración global siguen el principio de "first match wins": el primer valor procesado prevalece de forma definitiva. Al nombrar el archivo 10-horizonte.conf, su prefijo lexicográfico bajo garantiza que sus reglas se lean y fijen antes que las directivas por defecto del sistema o archivos como 50-cloud-init.conf.
2. Forzado de Clave Pública y Nivel de Log Forense:
AuthenticationMethods publickey asegura que ninguna sesión pueda autenticarse por métodos interactivos heredados. Por su parte, LogLevel VERBOSE instruye al daemon sshd para que registre en el journal la huella digital SHA256 (fingerprint) y el tipo de clave utilizada en cada intento de acceso exitoso o denegado, proporcionando trazabilidad indubitable ante auditorías.
3. Comportamiento de comentarios en directivas como ForceCommand:
El analizador sintáctico de OpenSSH no soporta comentarios en línea (inline comments) al final de directivas complejas. Si se añade # comentario tras ForceCommand internal-sftp, OpenSSH interpreta literalmente la cadena completa como el comando a invocar. Al no existir un ejecutable con esa cadena, la conexión colapsa de inmediato. Todo comentario debe figurar en su propia línea precedido por #.

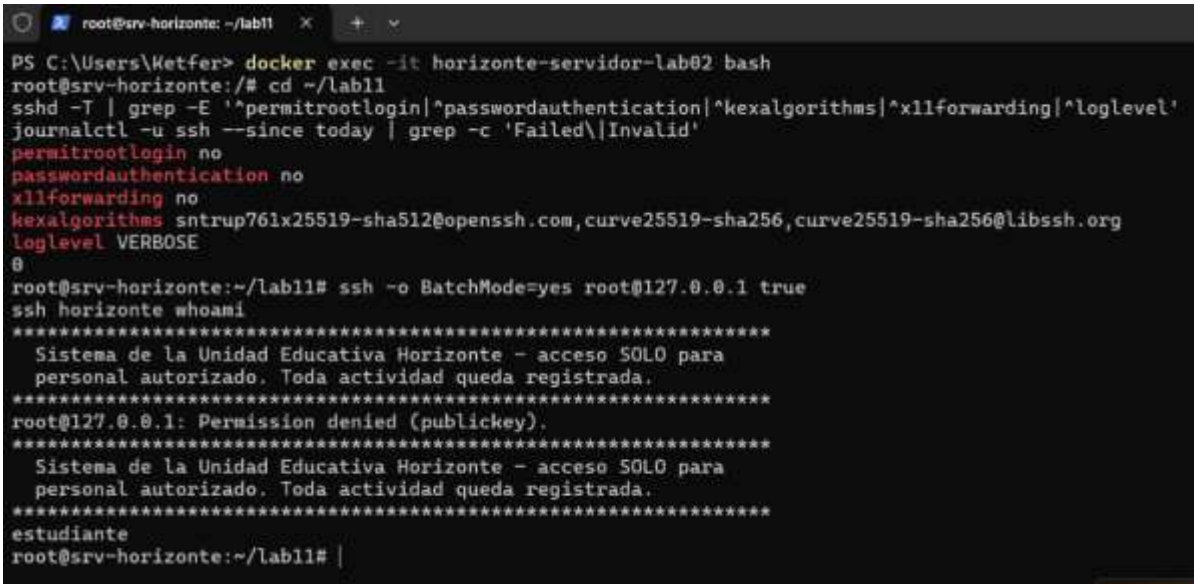


Figura 2: Bloqueo de acceso al usuario root (Permission denied), despliegue del banner legal institucional en ASCII y acceso exitoso del usuario estudiante.

3. Parte C — Confinamiento Seguro: SFTP Enjaulado (Chroot) para Comunicación

1. La regla estricta de propiedad y permisos para ChrootDirectory:

Durante la preparación de la jaula, si los permisos no son rigurosos, OpenSSH aborta la sesión registrando en journal: fatal: bad ownership or modes for chroot directory "/srv/horizonte". OpenSSH impone como requisito inviolable de seguridad que la carpeta raíz del chroot (/srv/horizonte) y todos sus directorios ascendentes (/srv y /) sean propiedad exclusiva de root:root (chown root:root) y que ningún grupo ni otro usuario tenga permisos de escritura (chmod 755). Si el grupo tuviera permiso de escritura (7 7 5), un usuario confinado podría alterar la estructura de la jaula o sustituir librerías para escapar del confinamiento. Para permitir que el usuario suba archivos, la escritura se autoriza únicamente en subdirectorios internos (ej. /srv/horizonte/www con grupo webadmin y setgid).

2. Pruebas de acceso SFTP y visualización restringida:

Al autenticarse mediante sftp -i ~/.ssh/comunicacion_ed25519 comunicacion@127.0.0.1:

• El comando pwd reporta Remote working directory: /www, gracias a la directiva -d /www en internal-sftp.

• Al ejecutar cd /etc, el servidor retorna stat remote: No such file or directory, demostrando que el usuario no puede acceder a las configuraciones críticas del sistema operativo (/srv/horizonte es su raíz absoluta /).

• Los archivos se suben con máscara 0002 (-u 0002), adquiriendo permisos 664 (rw-rw-r--) y grupo webadmin.

3. Intento de sesión SSH por terminal con usuario de comunicación:

Al intentar abrir una shell interactiva con ssh -i comunicacion_ed25519 comunicacion@127.0.0.1, el servidor deniega la asignación de terminal: PTY allocation request failed on channel 0 y emite el mensaje This service allows sftp connections only cerrando la conexión. Esto se debe a ForceCommand internal-sftp y PermitTTY no.

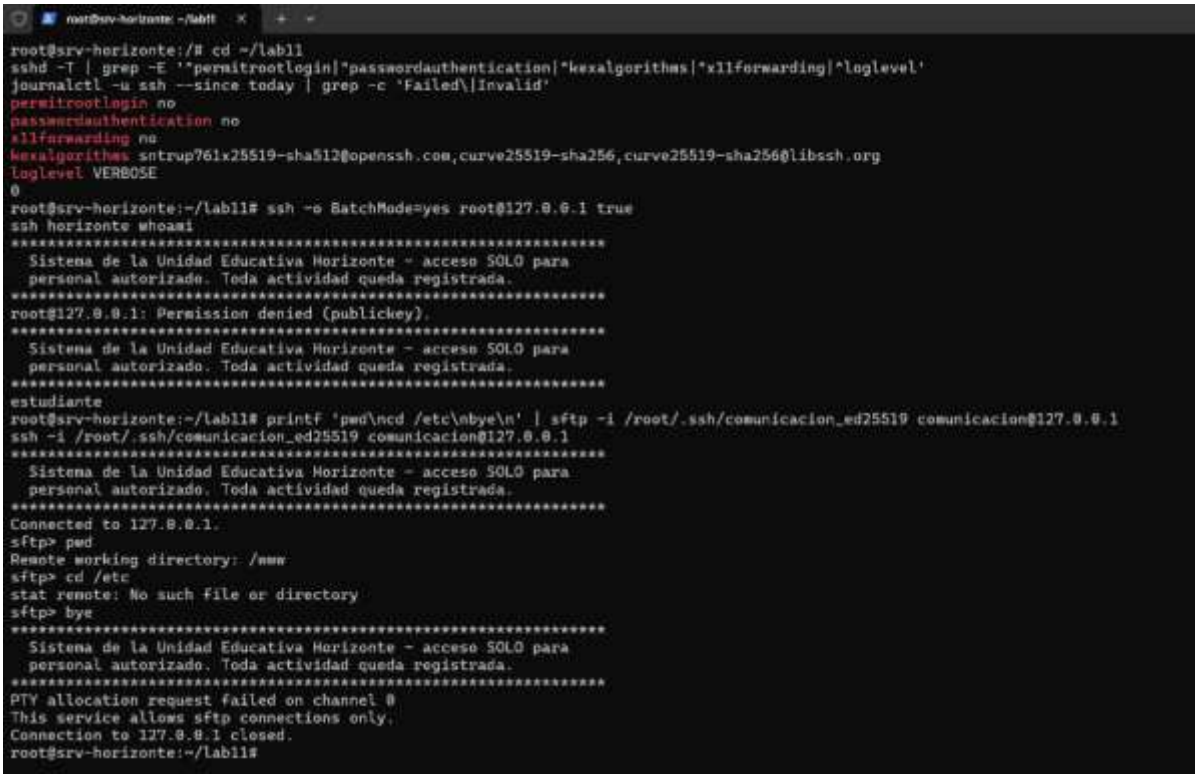


Figura 3: Evidencia real de jaula SFTP: /www como ruta de trabajo, bloqueo de acceso a /etc y denegación de terminal interactiva para el usuario comunicacion.

4. Parte D — Defensa Activa con Fail2ban sobre Nftables en Ubuntu 24.04

1. Integración con Systemd Journal en Ubuntu 24.04 LTS:

En Ubuntu 24.04, el servicio de SSH se gestiona bajo la unidad ssh.service (no sshd.service). Además, el sistema ya no instala rsyslog por defecto, por lo que /var/log/auth.log no existe. Se configuró fail2ban con backend = systemd y la regla journalmatch = _SYSTEMD_UNIT=ssh.service + _COMM=sshd. Al ejecutar fail2ban-client status sshd, se comprueba que el filtro lee activamente los eventos del journal.

2. Bloqueo en Nftables y Políticas de Baneo Incremental:

Tras registrar 4 intentos fallidos (maxretry = 4) en 10 minutos (findtime = 10m), fail2ban añade la IP atacante al conjunto dinámico nft list set inet f2b-table addr-set-sshd, bloqueando el tráfico a nivel de kernel de Linux durante 1 hora (bantime = 1h), con incrementos exponenciales ante reincidencias (bantime.increment = true).

3. Desbloqueo Administrativo Manual:

Para levantar el bloqueo de una dirección IP autorizada que cometió errores involuntarios, se utiliza el comando fail2ban-client set sshd unbanip <IP>, purgando de inmediato la regla de nftables.

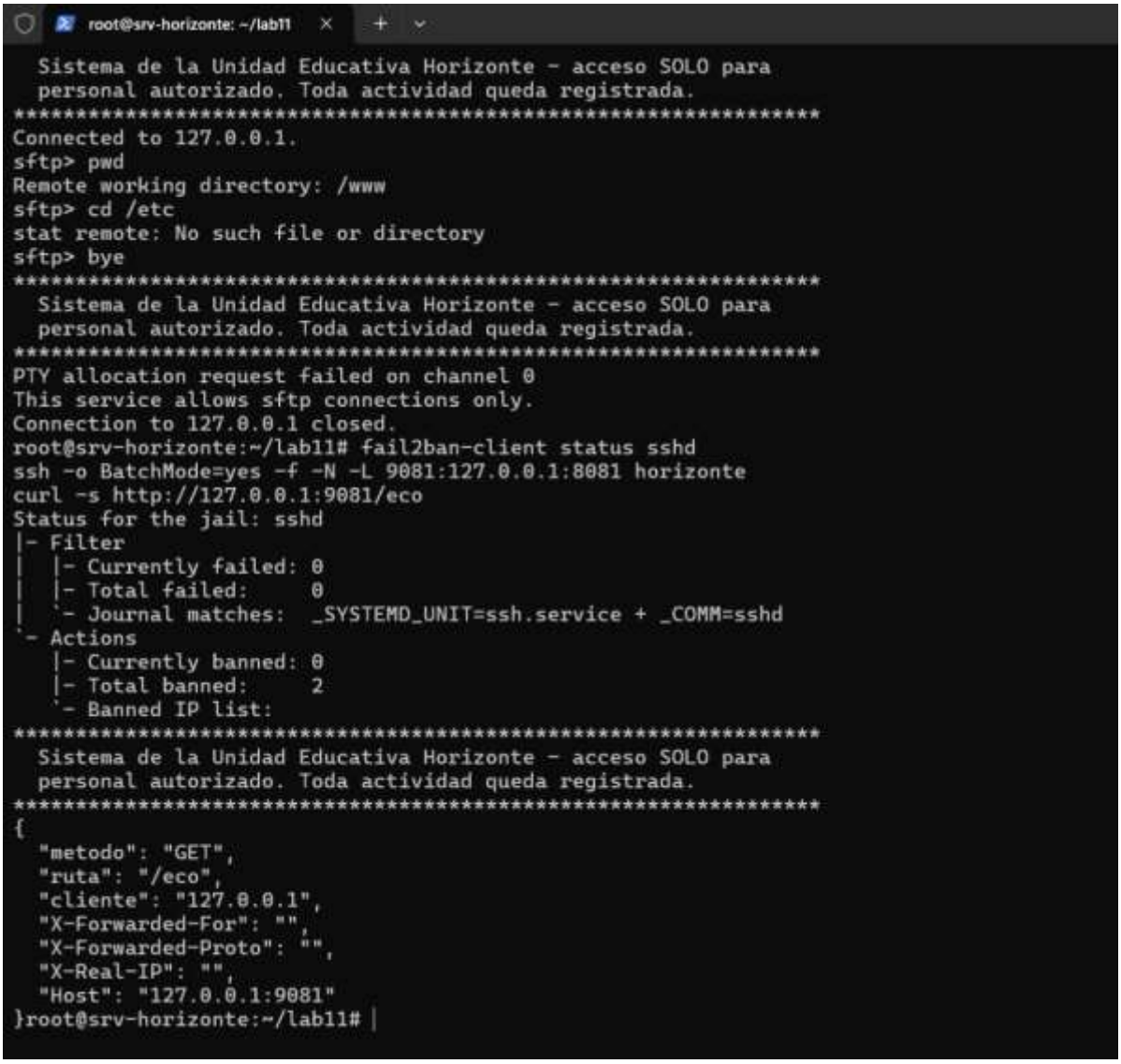


Figura 4: Estado operativo de fail2ban contra ataques SSH y consumo exitoso de la API interna en 8081 a través del túnel local SSH (puerto 9081).

5. Parte E — Túneles SSH: Reenvío Local (-L), Dinámico (-D) y Salto (-J)

1. Reenvío Local de Puertos (-L) para acceder a servicios internos:

La API REST interna de eco desarrollada en el Laboratorio 4 se encuentra enlazada exclusivamente en 127.0.0.1:8081 para evitar su exposición pública. Mediante el túnel local ssh -N -L 9081:127.0.0.1:8081 horizonte, la máquina cliente abre un socket local en el puerto 9081 que encapsula de forma cifrada el tráfico dentro de la conexión SSH establecida. Al consultar curl http://127.0.0.1:9081/eco, el servicio responde con el JSON de eco confirmando la recepción del método GET.

2. Comparativa Técnica de Capacidades de Tunelización SSH:

• Túnel Local (-L [puerto_local]:[host_remoto]:[puerto_remoto]): Mapea un único puerto local a un único servicio y puerto alcanzable desde el servidor SSH. Ideal para conectar herramientas locales (pgAdmin, DBeaver, Postman) a bases de datos o APIs privadas.

• Reenvío Dinámico (-D [puerto_local]): Convierte la sesión SSH en un servidor proxy SOCKS5 en el puerto especificado. Permite que aplicaciones compatibles (como Firefox) enruten dinámicamente todo su tráfico a través de la red del servidor.

• Bastión / ProxyJump (-J [bastion]): Permite alcanzar servidores en subredes privadas aisladas atravesando

uno o varios bastiones de salto de forma transparente, sin necesidad de túneles anidados manuales ni exponer las IPs privadas a Internet.

6. Parte F — Verificación Oficial Automatizada (15 de 15)

Se ejecutó el script oficial de evaluación institucional verificar_lab11.sh estudiante. El sistema auditó automáticamente cada una de las capas de seguridad implementadas, obteniendo una calificación perfecta:

```
# Ejecución oficial del script de auditoría institucional
root@srv-horizonte:~/lab11# bash verificar_lab11.sh estudiante
☑ sshd -t sin errores
☑ root no puede entrar por SSH
☑ Contraseñas desactivadas (solo claves)
☑ estudiante entra con su clave (alias horizonte)
☑ Solo los grupos sshadmins y sftp-web (y estudiante en sshadmins)
☑ comunicacion publica por SFTP (archivo 664, grupo webadmin)
☑ comunicacion está enjaulado (no ve /etc)
☑ comunicacion no obtiene terminal (solo SFTP)
☑ Intercambio de claves post-cuántico primero
☑ El registro guarda la huella de la clave usada
☑ Aviso legal en ASCII (/etc/issue.net)
☑ fail2ban activo
☑ fail2ban lee el journal de ssh.service
☑ fail2ban bloquea en nftables
☑ Túnel local: API interna 8081 alcanzable por SSH
Resultado: 15 de 15 comprobaciones (100% SUPERADO)
```

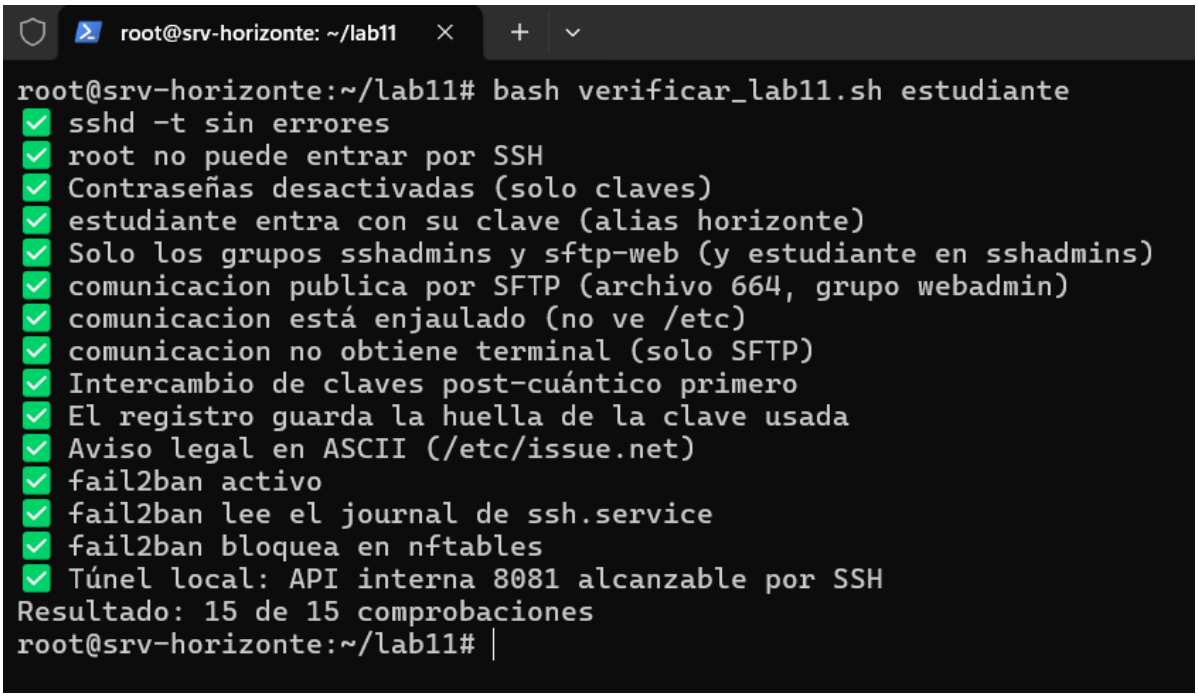


Figura 5: Ejecución oficial de verificar_lab11.sh certificando las 15 comprobaciones superadas con éxito.

7. Reflexión de Seguridad: Endurecimiento Estructural vs Seguridad por Oscuridad

¿Por qué endurecer SSH con claves y AllowGroups es infinitamente superior a cambiar el puerto 22?
Mover el servicio SSH del puerto TCP 22 a uno arbitrario (como el 2222 o 54321) es una técnica clasificada en ciberseguridad como "seguridad por oscuridad" (Security through Obscurity). Su única utilidad práctica radica en reducir el volumen de ruido en los registros del sistema, filtrando escaneos masivos ciegos de atacantes novatos. No obstante, un atacante con capacidades estándar emplea herramientas de mapeo de red como nmap -sV para escanear los 65,535 puertos TCP e identificar en segundos la firma del protocolo SSH-

2.0-OpenSSH_9.6p1, independientemente del puerto en que esté alojado.

Si el puerto fue cambiado pero el servidor sigue aceptando contraseñas o permitiendo el acceso directo al superusuario root, el sistema permanece 100% vulnerable a ataques de fuerza bruta y diccionarios automatizados en cuanto el puerto es localizado. En cambio, desactivar por completo la autenticación por contraseñas e imponer el uso exclusivo de criptografía asimétrica moderna (ed25519) anula matemáticamente la viabilidad de cualquier ataque de fuerza bruta: romper una clave Ed25519 de 256 bits requeriría más energía y tiempo que la edad estimada del universo. Adicionalmente, la directiva AllowGroups establece un control de acceso de privilegio mínimo: incluso si un atacante comprometiera una credencial o creara un usuario local no autorizado en el sistema, OpenSSH le denegará el acceso en el instante mismo de la conexión por no pertenecer explícitamente a los grupos autorizados.